

MICRODUCK PHYSICAL AI MASTERCLASS

PHASE 4 HANDOUT

Module 4: Brain Surgery: Actor Isolation, Hardware Clamps, ONNX Export & Temporal Memory

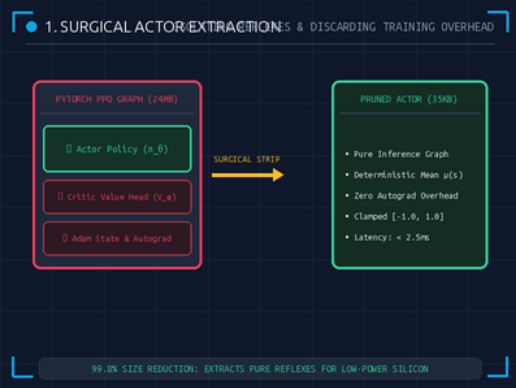
Edge Inference

PRUNED SIZE: 35 KB (99.8% Drop)

INFERENCE: 2.5ms on RK3566

SAFETY: torch.clamp()

BUFFER: deque(maxlen=4)



1. Surgical Actor Extraction

REFLEX ISOLATION & 99.8% PRUNING

Pruning Training Overhead: The 24MB PyTorch checkpoint contains the Critic value network $V_{\pi}(s)$, Adam optimizer state, and backward autograd graphs. We extract only the 35KB forward Actor $\pi_{\theta}(a|s)$.

The 12-Year-Old Analogy: When you take a piano exam, your teacher doesn't sit on your lap. You leave the teacher at home and only bring your muscle memory.

- **Actor Model:** Extracts deterministic mean $\mu(s)$ policy graph for direct joint actuation.
- **Weight Freezing:** Discards gradient tracking ('requires_grad=False'), eliminating dynamic memory allocations.
- **Silicon Size:** Reduces model footprint from 24,000 KB down to a lightweight 35 KB binary.



2. Hardware Safety Clamping in Silicon

TORCH.CLAMP() BOUNDS & GEAR PROTECTION

Mathematical Inviolability: Unbounded neural outputs (>1.0) can strip physical nylon servo gears. By baking 'torch.clamp(a, -1.0, 1.0)' into the computational graph, motor safety is guaranteed in silicon.

The 12-Year-Old Analogy: Bumper rails on a bowling lane that physically prevent the ball from bouncing into the next lane, no matter how hard you throw it.

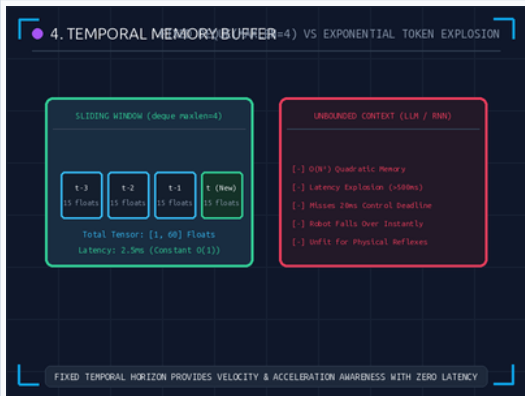
- **Silicon Clamping:** Compiled as a native ONNX 'Clip' operator executed directly on the NPU.
- **Range Guarantee:** Outputs are mathematically bounded to $[-1.0, 1.0]$ before reaching the PWM bus.
- **Failsafe Redundancy:** Prevents runaway policy gradients from destroying \$399 physical hardware.



■ 3. ONNX Computational Graph

CROSS-PLATFORM IR &
2.5MS EXECUTION

- Standardized Tensor Graph:** Compiles PyTorch weights into an ONNX Opset 17 computational DAG (Gemm $\rightarrow$ ReLU $\rightarrow$ Clip), executable via ``onnxruntime`` across any CPU/NPU hardware without Python.
- The 12-Year-Old Analogy:** Translating an English recipe into universal cooking symbols that any chef in any country can read and execute instantly.
- **Execution Speed:** Runs full 60-float inference in <2.5ms on Rockchip RK3566 quad-core ARM.
 - **Deterministic Pipeline:** Eliminates Python Global Interpreter Lock (GIL) and runtime garbage collection.
 - **Opset 17 Standard:** Portable representation deployable across C++, Rust, or embedded Linux kernels.



■ 4. Temporal Memory & Sliding Buffer

FIXED DEQUE(MAXLEN=4)
VS TOKEN EXPLOSION

- Fixed Memory Horizon:** Balancing requires velocity and acceleration awareness. A fixed sliding window ``deque(maxlen=4)`` buffers 4 frames $\times$ 15 sensors = 60 floats with constant $O(1)$ memory.
- The 12-Year-Old Analogy:** Unlike ChatGPT which grows huge and slower the more you talk, the duck's memory is a 4-frame conveyor belt where old frames quietly drop off.
- **Constant O(1) Memory:** Prevents LLM-style $O(N^2)$ quadratic token memory growth and latency spikes.
 - **State Dimensions:** Produces flat `[1, 60]` input tensor capturing instantaneous joint momentum.
 - **Strict 50Hz Budget:** 2.5ms inference + 17.5ms cadence sleep = perfectly steady 20.0ms heartbeat.

■ PHASE 4 SILICON DEPLOYMENT SUMMARY & RUST DAEMON HANDOFF

- **Reflex Model:** Extracted 35KB ONNX policy containing pure clamped walking muscle memory.
- **Real-Time Cadence:** Predictable 2.5ms inference guarantees zero timing jitter on edge hardware.
- **Next Module:** Deploy the ONNX binary into Phase 5: The Rust Nervous System (``robotd`` & ``robotctl``).

■ PHASE 4 KNOWLEDGE CHECK

ASSESSMENT

Test your understanding of Model Extraction, Clamping, ONNX Compilation, and Temporal Memory.

5 Questions

■ Q1: Actor Policy Extraction

MULTIPLE CHOICE

Why do we extract only the Actor network and discard the Critic?

- A. Because the Critic is written in a different programming language.
- B. The Critic is only needed to score actions during training; the Actor contains all walking reflexes.
- C. The Critic network causes the robot to walk backwards.

■ Q2: Silicon Clamping Protection

MULTIPLE CHOICE

What is the primary danger of deploying an ONNX policy without torch.clamp()?

- A. Runaway neural outputs (>1.0) can command excessive speed, stripping physical gears.
- B. The robot's battery will charge too quickly.
- C. The WiFi antenna will disconnect.

■ Q3: ONNX Edge Execution

MULTIPLE CHOICE

What is the advantage of compiling PyTorch models to ONNX for edge robotics?

- A. ONNX allows the robot to fly.
- B. ONNX represents pure math graphs, executing in <2.5ms without heavy Python overhead.
- C. ONNX turns the camera into an IMU.

■ Q4: 50Hz Timing Discipline

MULTIPLE CHOICE

Why does our 50Hz control loop explicitly sleep for 17.5ms after a 2.5ms inference?

- A. To maintain a perfectly steady 20ms heartbeat and prevent motor timing jitter.
- B. To allow the CPU to cool down to 0 degrees.
- C. Because Linux cannot run for more than 3ms.

■ Q5: Sliding Temporal Memory

MULTIPLE CHOICE

Why does deque(maxlen=4) avoid the 'token explosion' problem of LLMs?

- A. It deletes the robot's brain every second.
- B. It holds a fixed 60-float buffer, ensuring constant O(1) memory and predictable 2.5ms latency.
- C. It compresses video into text files.

Answer Key: 1-B, 2-A, 3-B, 4-A, 5-B (1-B: Actor reflexes; 2-A: Gear strip prevention; 3-B: Fast math graph; 4-A: Precise 20ms cadence; 5-B: Constant O(1) buffer)